Mahogany - Updates

Reliability + Coverage Upgrade (Task/Feature List / PRD Draft)

1) Goal / desired outcome

Improve the end-to-end reliability, freshness, and relevance of Mahogany's regulatory intelligence pipeline so that:

- Ingestion consistently pulls real content (not junk/empty pages).
- Agents can reliably reach and parse sources without getting blocked.
- The knowledge graph is comprehensive for the user's domain and interests.
- The news feed + email digest capture the majority of what the user cares about (target: ~80% recall) while minimizing irrelevant "slop."
- Semantic + natural-language chat returns the results users expect.
- Users can track specific entities (product codes, regs, legislation) and receive dependable updates.
- The UI/design system feels polished and credible.

2) Scope (what needs to be updated)

A) Source ingestion quality + link integrity

1. Link health & extraction validation

- Add automated link-checking for all configured sources (API endpoints, RSS URLs, scraped pages).
- Detect and flag common failure modes:
    - 404/410, redirect loops, forbidden/blocked (403), rate limiting (429), timeouts.
    - "Soft failures" where the page loads but content is empty, boilerplate-only, or irrelevant.

- For each ingested item, store extraction diagnostics:
  - HTTP status, final resolved URL, response time, content length, extracted text length, main-content ratio, and parser used (RSS/API/Firecrawl/etc.).

1. “Junk” prevention / content sanity checks

- Create a quality gate before content is accepted into the pipeline:
  - Minimum extracted text threshold.
  - Duplicate detection (hashing + similarity).
  - Boilerplate detection (nav-heavy, cookie banners, login pages).
  - Language detection + expected-domain checks.
- Route failed items into an “ingestion exceptions” queue with reason codes and suggested remediation.

1. Fallback strategies when extraction fails

- Implement layered fallback per source type:
  - If RSS fails → attempt HTML fetch + extraction.
  - If Firecrawl fails/returns thin content → attempt alternate extraction mode or direct fetch + readability parsing.
  - If API returns partial data → retry, then degrade gracefully (store metadata, mark content incomplete).
- Add “source change detection” (structure changed, feed schema changed) and auto-alert when failure rate spikes.

B) Agent reliability: access, blocking, and observability

1. Connectivity and block detection

- Add agent “ping” checks to confirm sources are reachable and not blocked.
- Track bot mitigation signals (CAPTCHA pages, challenge pages, sudden 403 spikes).
- Provide recommended mitigation knobs (user-agent rotation, backoff, caching, alternate endpoints) and document which are allowed/appropriate.

1. Agent run monitoring

- For each agent run, record:
  - Inputs, sources touched, outputs produced, failures, and latency.
- Add a dashboard (or internal page) that surfaces:
  - Success rate by source
  - Freshness lag
  - Top failing sources + reason codes
  - Extraction quality distribution

C) Freshness / real-time updates

1. Define freshness targets by source class

- APIs: near real-time or hourly depending on constraints.
- RSS: frequent polling (e.g., 10–30 min) with backoff.
- Scraped pages: scheduled by importance + volatility.

1. Scheduling + incremental updates

- Move to incremental pulls where possible (ETags, Last-Modified, feed GUIDs, API cursors).
- Implement retries with exponential backoff + jitter.
- Store “last successful ingestion” timestamps and alert when stale beyond SLA.

D) Knowledge graph completeness (regulatory breadth + user relevance)

1. Coverage strategy

- Establish a “source map” and coverage goals (initial target: map to 100 sources).
- Ensure coverage includes:
  - Core regulators (FDA, EMA, etc.)
  - Standards bodies and guidance repositories

    - Legislative trackers
    - Enforcement / recalls / safety communications
    - Relevant industry updates tied to user profile

1. Normalization + entity linking

- Normalize incoming content into consistent entities:
    - Product codes, regulation IDs, guidance docs, legislation bills, standards, manufacturers, devices, indications, etc.
- Improve deduplication and linking logic so the graph aggregates across sources rather than fragmenting.

E) Feed + email digest: relevance, recall, and “not slop”

1. Define success metrics

- Target recall: capture ~80% of “things the user cares about.”
- Precision guardrails: minimize irrelevant items; cap repetitive near-duplicates.
- Add lightweight user feedback loops: “relevant / not relevant” signals that feed ranking.

1. Ranking and filtering based on user profile

- Ensure the profile acts as an intelligent filter:
    - Doesn’t cut too aggressively (missing key updates)
    - Doesn’t let through low-signal content
- Implement tiering:
    - Must-see alerts (high-impact regulatory changes)
    - Weekly digest (contextual, lower urgency)
    - Optional exploratory items (clearly labeled)

F) Search + semantic / NL chat quality

1. Validate retrieval pipeline end-to-end

- Confirm the chat experience can:
  - Retrieve the correct underlying documents/graph nodes
  - Cite the right sources and timestamps
  - Answer with the expected level of specificity (not generic summaries)

1. Add evaluation tests

- Build a small set of “golden questions” tied to real user needs:
  - “What changed recently for [regulation/product code]?”
  - “Show updates affecting [device category] in the last [time window].”
- Measure response correctness, citation accuracy, and coverage.

G) User tracking: product codes, regs, legislation + update delivery

1. Tracking objects

- Ensure users can explicitly track:
  - Product codes, regulation identifiers, legislative bills, guidance documents, keywords/entities.
- Track subscriptions with clear state:
  - Active, paused, frequency preferences, alert thresholds.

1. Delivery guarantees

- Confirm users receive updates reliably:
  - Email digest sends on schedule
  - High-priority alerts are delivered
  - Backfill behavior when ingestion was temporarily down

H) UX / design system polish

1. Visual hierarchy and credibility

- Upgrade typography, spacing, and component consistency.

- Improve information hierarchy in feed/digest views:
  - What changed, why it matters, impacted entities, and source credibility.

1. Product feel improvements

- Faster perceived performance, clearer loading/empty/error states.
- Better “trust” indicators (timestamps, sources, last updated, extraction status)

---

3) Explicit requirements from the checklist

- Ensure links are not broken and API calls, RSS feeds, and Firecrawl are truly extracting content (not junk).
- Ensure agents can extract data from sources (can ping them; not blocked by websites).
- Ensure content is as real-time as possible and regularly updated.
- Ensure the knowledge graph pools comprehensive regulatory data, especially data relevant to the user.
- Ensure comprehensive content comes through the news feed and email digest (target: ~80% of what the user cares about).
- Ensure the profile acts as an intelligent filter for both feed and digest (not too restrictive; not slop).
- Ensure AI semantic + NL chat works and returns expected results.
- Ensure users can track product codes, regulatory changes, and legislation and receive regular updates.
- Have a fallback and/or be able to locate when a link goes missing or a source changes.
- Make the design system/UI significantly sleeker and more polished (visual hierarchy, overall quality).

4) Coverage milestone

- Map to 100 data sources (tracked with ownership, ingestion method, freshness SLA, and current health status).